

13.2 Self-Consistency and Tree-of-Thoughts

Sample several traces and vote. Search a tree of partial thoughts when one path is not enough.

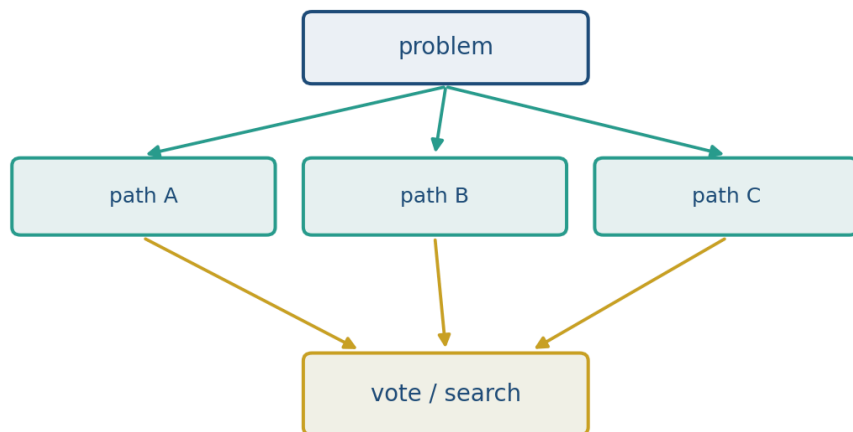
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

One chain is a sample, not a proof. **Self-consistency** asks for several traces and **votes**. **Tree-of-thoughts** (ToT) searches among partial plans instead of committing to a single left-to-right story.

1. Majority vote over traces

Sample k independent CoT traces (temperature > 0). Parse an answer from each. Take the majority. The hope: wrong paths disagree; right paths land on the same number.

Self-consistency votes; Tree-of-Thoughts searches partial steps



This is test-time compute. You pay k forwards. Report k and the parser (last integer, boxed span, regex). Voting on the **answer**, not on the wording of the steps, is the usual rule. Lab 13 does that vote on constructed strings. Plot accuracy against k if you use this in a project; the curve usually flattens.

Self-consistency does not fix a shared bug. If every trace uses the same wrong formula, the majority is still wrong.

2. Search, not just i.i.d. samples

ToT treats a “thought” as a node: a subgoal, a candidate equation, a next move in a game. You expand a few children, **score** them (a prompt, a heuristic, or a checker), and keep a beam or run BFS/DFS. That is search with an LM as the proposal distribution. Greedy CoT is a tree of depth equal to one chain and width 1. ToT buys width.

ReAct (Week 14) is a different control loop: the node can be a **tool call**, not only more text. Do not mix the names in the report. Vote versus tree versus tools are three knobs.

A cheap ToT scorer you can actually run in this course: last integer equals a known subgoal, a unit test, or `calc` from Week 14. An unconstrained “does this thought look good?” prompt is another noisy LM, not a checker.

3. What to measure

Accuracy versus k is the self-consistency plot. For ToT, accuracy versus expansion budget (nodes scored). Always include a greedy CoT baseline at $k = 1$. Cost belongs in the table: tokens, or wall time on the hardware you used.

4. Teaching this note

About **35 minutes** at the board, then **~10 minutes** of video.

- **0–12 min.** Temperature > 0 , k traces, vote on the parsed answer. Tie-break rule.
 - **12–22 min.** ToT as width: expand, score, keep a beam. Contrast with ReAct (tools).
 - **22–33 min.** Worked majority vote on three traces (Lab 13 numbers).
 - **Then** play Karpathy Deep Dive **1:46:56–2:01:11** (models need tokens to think). Pause on distributing hard work across tokens; then say: self-consistency is k of those traces, ToT is a tree of them.
-

5. Worked example

Problem: $23 + 19$, gold 42. Three traces, answers only (Lab 13 style):

trace	parsed answer	steps (sketch)
A	42	$20 + 19 = 39$, $39 + 3 = 42$
B	32	$23 + 19 = 32$ (digit bug)
C	42	carry arithmetic to 42

Vote on **answers**: 42, 32, 42. Majority is 42 (2 of 3). $k = 1$ using only B would have scored 32 and missed. Self-consistency caught the disagreement.

Tie example: answers 42, 32, 32. Majority is 32, which is **wrong**. Voting is not a proof; it is a noise reducer when errors are **diverse**.

Shared-bug example: all three use $23 + 19 = 32$. Vote is 32. $k = 3$ does not help. A checker (note **13.3**, or a `calc` tool in Week 14) would.

ToT cartoon for $6 \times 7 + 5$: nodes “do multiply first” vs “do add first.” A cheap scorer (PEMDAS, or `eval`) kills the add-first branch before you decode a full wrong chain.

6. Where students get stuck

- Sampling at temperature 0 and wondering why all k traces match. You need diversity for a vote.
 - Voting on the **wording** of the steps. Parse an answer; majority on that.
 - Calling ToT “just CoT with more words.” ToT scores **partial** nodes and prunes.
-

7. Video

Watch [Andrej Karpathy, Deep Dive into LLMs like ChatGPT](#), **1:46:56–2:01:11**.

Pause on “models need tokens to think”: extra tokens are extra compute. This course’s self-consistency is k independent traces plus a vote; ToT is search over partial thoughts. Optional: **2:14:42–2:27:47** (RL that explores many solutions) if you want a later-week bridge. Note **13.3** uses a different chapter of the same URL.

8. Practice

1. Why does self-consistency need temperature greater than 0?
2. Give one failure that majority vote cannot catch.
3. ToT scores partial thoughts. Why is a cheap checker (arithmetic, a unit test) a better scorer than another unconstrained prompt, when you have one?
4. Traces vote 47, 65, 47, 47. What is the majority, and what is k ? Gold is 47. Did vote beat a first-trace-only policy if the first trace was 65?
5. Answers 408, 408, 238 for 17×24 . Write the vote. If you instead vote on full step strings (all different wording), what goes wrong?